

New York University Tandon School of Engineering
Computer Science and Engineering

CS-GY 6033: Written Homework 5.
Due Thursday, April 16, 2026, 11:59pm.

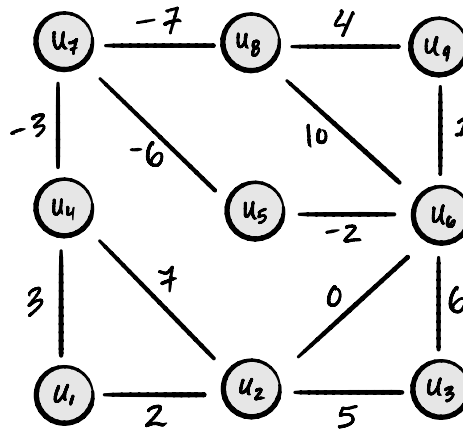
Collaboration is allowed on this problem set, but solutions must be written-up individually. If we suspect that you copied your solution directly from AI, we will set up a meeting with you where you will need to explain your solution. If you fail to do so, you will receive zero points for the homework.

Problem 1

In this problem you will be asked to list the edges in the minimum spanning tree of the graph below in the order that they are added by either Prim's algorithm or Kruskal's algorithm.

In order to simplify grading, please write an edge with the *smaller* node first. For example: (u_3, u_7) instead of (u_7, u_3) .

Also, when writing an edge, make sure to write the edge as a pair of nodes, not the weight of the edge. Thanks!



- Suppose Prim's algorithm is run on the above graph, using node u_1 as the starting node. List the edges of the resulting minimum spanning tree computed in the order that they are added by the algorithm.
- Suppose Kruskal's algorithm is run on the graph above. List the edges of the resulting minimum spanning tree in the order that they are added by the algorithm.

Problem 2

Suppose we are given a undirected weighted graph $G = (V, E; w)$, where $w : E \rightarrow \mathbb{R}$ is the map to assign a weight $w(e)$ to each edge $e \in E$. Let T^* be a minimum spanning tree of G .

- Now let $G_1 = (V, E; w_1)$ be a modified version of G : In particular, G_1 share the same vertex set and edge set as G . The only difference lies in the edge weights, where for any edge $e \in E$, the new weight doubles the previous weight $w(e)$; that is, $w_1(e) = 2w(e)$.
(True or False): The tree T^* is still a minimum spanning tree for G_1 as well. You do not need to provide a justification.
- Now let $G_2 = (V, E; w_2)$ be another modified version of G : In particular, G_1 share the same vertex set and edge set as G . The only difference lies in the edge weights, where for any edge $e \in E$, the new weight equals the previous weight $w(e) + 2$; that is, $w_2(e) = w(e) + 2$.
(True or False): The tree T^* is still a minimum spanning tree for G_2 as well. You do not need to provide a justification.

- c. Let $s \in V$ be a source node. Let $\hat{T}$ be a shortest path tree of G from the source s . Consider the modified graph G_2 defined above.
- (True or False): The tree $\hat{T}$ is a shortest path tree for G_2 from the source s as well.

Problem 3

In lecture, we saw that Kruskal's algorithm can be used to cluster a weighted graph. The name for this approach is *single linkage clustering*.

In a file named `slc.py`, write a function `slc(graph, d, k)` which accepts the following arguments:

- **graph**: a dictionary of lists representing an undirected graph.
- **d**: $d(u, v)$ is a dictionary mapping edges to their weights i.e., distance between node u and v .
- **k**: A positive integer describing the number of clusters which should be found.

The function should perform single linkage clustering using Kruskal's algorithm and it should return a **frozenset** of k **frozensets**, each representing a cluster of the graph.

Note: to implement Kruskal's algorithm, you'll need an implementation of a Disjoint Set Forest data structure.

Problem 4

Recall the greedy algorithm we discussed in the class for the maximum coverage problem. We showed that the greedy algorithm, in the worst case, finds a solution whose coverage is at least $(1 - \frac{1}{e}) \simeq 0.632$ percent of the optimal coverage **OPT**. At the heart of this guarantee is the loop invariant we talked about. That is

$$f(S_{i+1} - f(S_i)) \geq \frac{1}{k}(\text{OPT} - f(S_i)) \quad (1)$$

where S_i is the set computed by the algorithm at iteration i and $f(S_i)$ is the number of blue nodes S_i covers.

- Prove that, given the loop invariant, it holds that $\text{OPT} - f(S_i) \leq (1 - \frac{1}{k})(\text{OPT} - f(S_{i-1}))$.
- Using unrolling from the inequity in part (a) we can derive that $\text{OPT} - f(S_i) \leq (1 - \frac{1}{k})^i(\text{OPT})$. Show that this implies $f(S_k) \geq (1 - (1 - \frac{1}{k})^k) \cdot \text{OPT}$, for $i = k$.
- Prove that the loop invariant (1) is correct.

Problem 5

A hospital diagnostic lab processes blood test batches using a single automated analyzer. The machine can handle only one batch at a time, and once started, a batch must run to completion. There are n batches, all available at time 0. Each batch i has a processing time p_i (in minutes) and a promised reporting deadline d_i (minutes from now). If a batch finishes at time C_i , its lateness is defined as

$$L_i = C_i - d_i.$$

The goal is to find an order in which we should process the batches so that it minimizes the maximum lateness:

$$L_{\max} = \max_{i \in \{1, \dots, n\}} L_i.$$

Consider the following greedy scheduling rule: order all batches in nondecreasing order of deadlines d_i , and process them in that order.

A pair of adjacent jobs (i, j) in a schedule is called an *inversion* if job i is scheduled immediately before job j but $d_i > d_j$.

Let's prove that the above greedy scheduling rule minimizes $L_{\max}$:

- a. Argue that if a schedule is not ordered by nondecreasing deadlines, then it must contain at least one adjacent inversion.
- b. Consider an adjacent inverted pair (i, j) , and let t be the start time of job i . Write expressions for the completion times and lateness values of i and j before the swap, and after swapping their order.
- c. Using the expressions from part (b), show that after swapping (i, j) , the maximum lateness of these two jobs does not increase.
- d. Explain why repeatedly eliminating inversions leads to a schedule ordered by deadlines, and conclude that the given rule is optimal.